

Naro Fashion - Instagram Integration & Newsletter System Guide

Table of Contents

1. [Overview](#)
 2. [Architecture Diagram](#)
 3. [Instagram Graph API Integration](#)
 - 3.1 [How It Works](#)
 - 3.2 [Data Flow](#)
 - 3.3 [Post Ordering Logic](#)
 - 3.4 [Cron Jobs \(Auto Sync & Token Refresh\)](#)
 - 3.5 [Code Structure](#)
 4. [Facebook Developer Account Setup](#)
 - 4.1 [Prerequisites](#)
 - 4.2 [Step-by-Step App Creation](#)
 - 4.3 [Generating Access Token](#)
 - 4.4 [Finding Instagram Business Account ID](#)
 - 4.5 [Exchanging for Long-Lived Token](#)
 5. [Environment Configuration](#)
 6. [Admin Dashboard - Instagram Management](#)
 - 6.1 [Sync from Instagram](#)
 - 6.2 [Pin/Unpin Posts](#)
 - 6.3 [Manual Post Creation](#)
 - 6.4 [Visibility Toggle](#)
 7. [Storefront Display](#)
 8. [Newsletter System](#)
 - 8.1 [Subscriber Management](#)
 - 8.2 [Email Campaign Compose](#)
 - 8.3 [Template Types](#)
 - 8.4 [Delivery Tracking](#)
 - 8.5 [Resending Failed Emails](#)
 9. [Database Models](#)
 10. [API Endpoints Reference](#)
 11. [Troubleshooting](#)
-

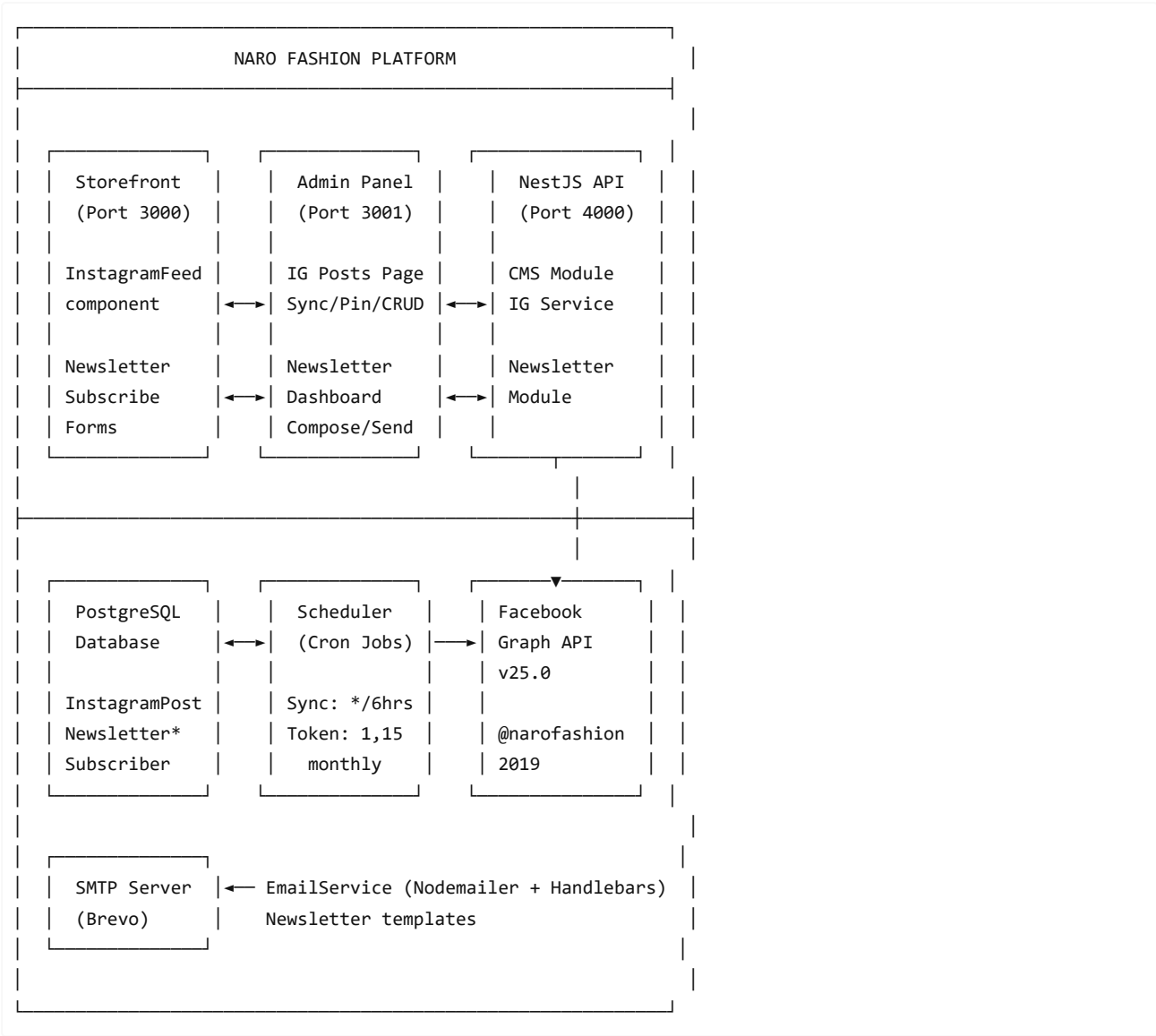
1. Overview

The Naro Fashion platform integrates with Instagram to automatically display the latest posts from the official @narofashion2019 account on the storefront. Additionally, a full newsletter/email campaign system allows admin to compose and send marketing emails to subscribers with delivery tracking.

Key Features:

- **Auto-sync** Instagram posts every 6 hours via Facebook Graph API
 - **Manual control:** Admin can add manual posts, pin/unpin, toggle visibility
 - **Smart ordering:** API posts (newest first) → Pinned posts → Manual posts
 - **Newsletter campaigns:** 4 template types with per-recipient delivery tracking
 - **New Arrivals auto-population:** Automatically includes products not yet sent in previous newsletters
 - **Failed email resend:** Track failures with reasons and retry
-

2. Architecture Diagram



3. Instagram Graph API Integration

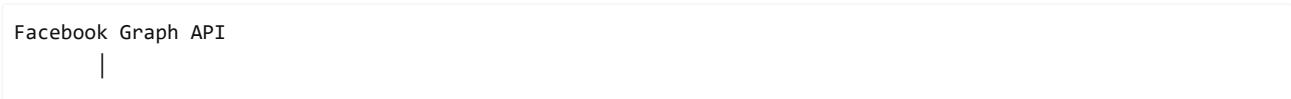
3.1 How It Works

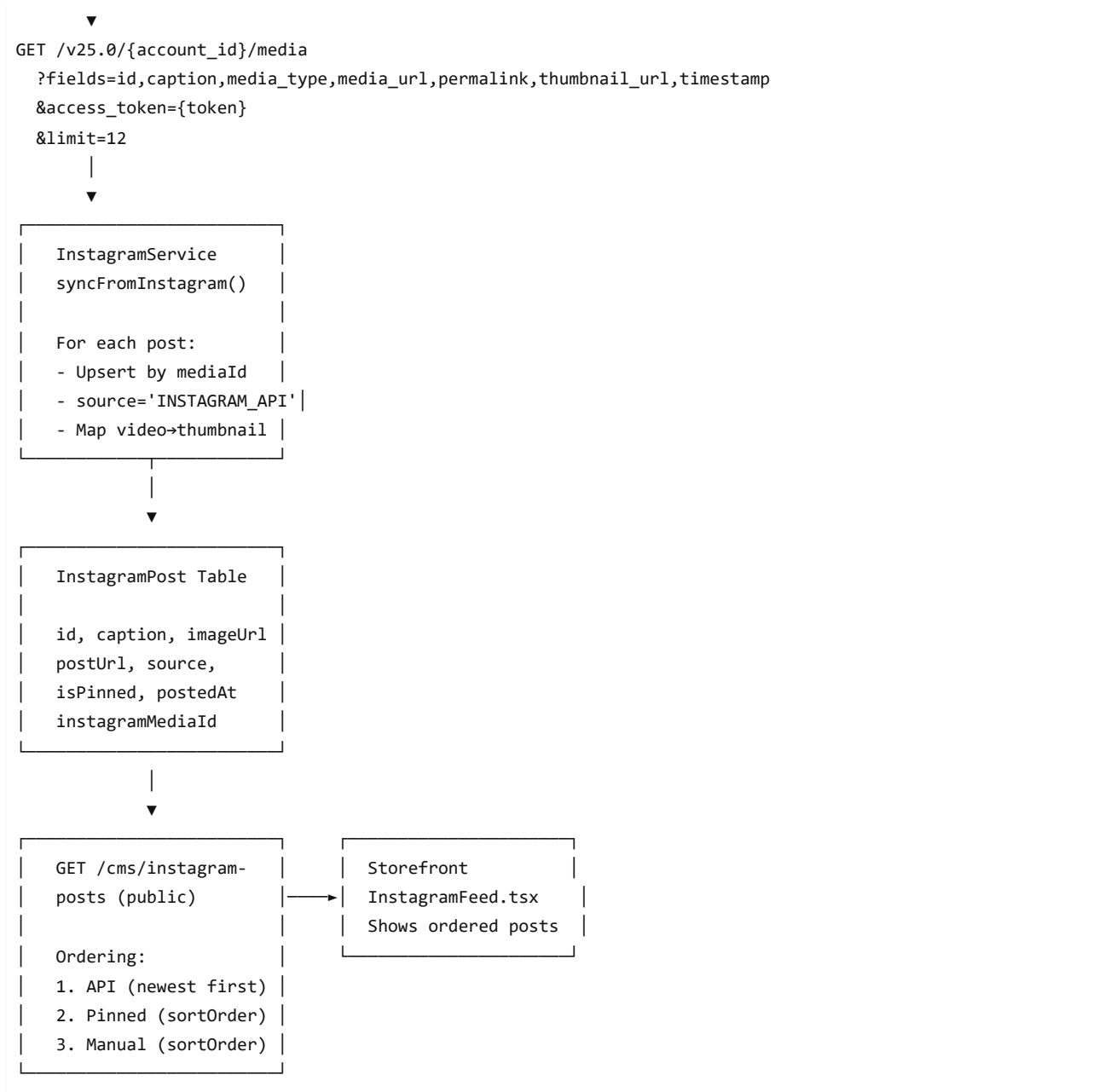
The integration uses the **Facebook Graph API v25.0** (not the deprecated Instagram Basic Display API) to fetch posts from the Instagram Business Account linked to the Facebook Page "nancyfashion2019".

Flow:

1. The `InstagramService` reads the access token and account ID from environment variables
2. Makes a GET request to `https://graph.facebook.com/v25.0/{ACCOUNT_ID}/media`
3. Fetches up to 12 most recent posts with fields: id, caption, media_type, media_url, permalink, thumbnail_url, timestamp
4. For each post, upserts into the `InstagramPost` database table using `instagramMediaId` as the unique key
5. Videos use `thumbnail_url` as the display image; images use `media_url`

3.2 Data Flow





3.3 Post Ordering Logic

When the storefront requests Instagram posts, the API returns them in this specific order:

Priority	Source	Sort By	Description
1st	INSTAGRAM_API	postedAt DESC	Latest posts from the real IG account appear first
2nd	Any with isPinned=true	sortOrder ASC	Admin-pinned posts appear next
3rd	MANUAL (not pinned)	sortOrder ASC	Admin-created manual posts appear last

Code location: `apps/api/src/cms/cms.service.ts` → `findActiveInstagramPosts()`

3.4 Cron Jobs

Two scheduled tasks run automatically:

Cron	Schedule	Job	Description
------	----------	-----	-------------

0 */6 * * *	Every 6 hours	instagram-sync	Syncs latest 12 posts from Instagram API
0 0 1,15 * * *	1st & 15th monthly	instagram-token-refresh	Refreshes the long-lived access token

Code location: `apps/api/src/scheduler/scheduler.service.ts`

Token refresh flow:

1. Takes the current token from `env/SiteSetting`
2. Calls Facebook's token exchange endpoint with App ID + App Secret
3. Receives a new long-lived token (60 days)
4. Stores the new token in the `SiteSetting` table for persistence
5. Next sync uses the refreshed token from `SiteSetting` (falls back to env var)

3.5 Code Structure

```
apps/api/src/
├── cms/
│   ├── cms.module.ts          # Registers InstagramService
│   ├── cms.service.ts        # Instagram CRUD + ordering logic
│   ├── cms.controller.ts     # Endpoints: sync, pin, CRUD
│   └── instagram.service.ts  # Facebook Graph API integration
├── scheduler/
│   └── scheduler.service.ts  # Cron jobs for sync + token refresh
└── notifications/
    └── templates/
        ├── newsletter.hbs      # Generic newsletter template
        └── newsletter-new-arrivals.hbs # Product grid template
```

4. Facebook Developer Account Setup

4.1 Prerequisites

- A Facebook account that manages the Naro Fashion Facebook Page
- An Instagram Professional/Business account (@narofashion2019) linked to that Facebook Page
- The Instagram account must be connected to the Facebook Page in Facebook Page Settings

4.2 Step-by-Step App Creation

1. Go to <https://developers.facebook.com/>
2. Click "Get Started" → complete developer registration if needed
3. Navigate to <https://developers.facebook.com/apps/> → "Create App"
4. **App details:** Enter app name (e.g., "Narofashion"), contact email
5. **Use cases:** Select "**Business messaging**" → check "**Manage messaging & content on Instagram**"
6. **Business:** Select your business portfolio (e.g., "Nancy Fashion")
7. **Review & Create:** Click "Create app"

4.3 Generating Access Token

1. In the app dashboard, go to **Use cases** → **Customize** the Instagram use case
2. Click "**API setup with Facebook login**" in the left sidebar
3. Under section 1, click "**Go to permissions and features**" to enable required permissions
4. Go to **Tools** → **Graph API Explorer**
5. Select your app ("Narofashion") and click "**User or Page**" → "**Get Token**" dropdown
6. Select the permissions:
 - `instagram_basic`
 - `pages_show_list`
 - `pages_read_engagement`

- `business_management`

7. Click **"Generate Access Token"**
8. Authorize the app and select your Facebook Page when prompted
9. Copy the generated token

4.4 Finding Instagram Business Account ID

In the Graph API Explorer, with the token generated:

1. Set the URL to: `me/accounts?fields=id,name,instagram_business_account`
2. Click **Submit**
3. The response contains your page with:

```
{
  "data": [{
    "id": "103693824395641",
    "name": "nancyfashion2019",
    "instagram_business_account": {
      "id": "17841418108905851" ← THIS IS YOUR ACCOUNT ID
    }
  }]
}
```

4.5 Exchanging for Long-Lived Token

The token from Graph API Explorer is short-lived (~1 hour). Exchange it for a long-lived token (60 days):

Request:

```
GET https://graph.facebook.com/v25.0/oauth/access_token
?grant_type=fb_exchange_token
&client_id={APP_ID}
&client_secret={APP_SECRET}
&fb_exchange_token={SHORT_LIVED_TOKEN}
```

Response:

```
{
  "access_token": "EAA9qKZANK1ncBQ...",
  "token_type": "bearer",
  "expires_in": 5184000
}
```

The `expires_in` value is in seconds (~60 days). The cron job automatically refreshes this before expiry.

5. Environment Configuration

Add these variables to `apps/api/.env` :

```
# Facebook Developer App
FACEBOOK_APP_ID="4338851449722487"
FACEBOOK_APP_SECRET="your_app_secret_here"

# Instagram Graph API
```

```
INSTAGRAM_ACCESS_TOKEN="EAA9qkZANK1ncBQ..." (long-lived token)
INSTAGRAM_BUSINESS_ACCOUNT_ID="17841418108905851"
```

Variable	Source	Expiry
FACEBOOK_APP_ID	App Settings → Basic	Permanent
FACEBOOK_APP_SECRET	App Settings → Basic	Permanent (can be reset)
INSTAGRAM_ACCESS_TOKEN	Token exchange endpoint	60 days (auto-refreshed)
INSTAGRAM_BUSINESS_ACCOUNT_ID	Graph API: me/accounts query	Permanent

6. Admin Dashboard - Instagram Management

6.1 Sync from Instagram

Location: Admin → CMS → Instagram Posts

The "Sync from Instagram" button triggers a manual sync. It calls `POST /api/v1/cms/instagram-posts/sync` which:

- Fetches latest 12 posts from the Graph API
- Upserts into database (new posts added, existing updated)
- Returns `{ synced: 12, errors: 0 }`

Posts from Instagram appear with a blue **"IG"** badge.

6.2 Pin/Unpin Posts

Any post (API or manual) can be pinned by clicking the **Pin** icon on hover. Pinned posts:

- Show a gold **"Pinned"** badge
- Appear after API posts but before manual posts on the storefront
- Are sorted by `sortOrder` among themselves

6.3 Manual Post Creation

Click "Add Post" to create a manual post with:

- Image URL (required)
- Caption
- Instagram post URL (optional)
- Likes count
- Sort order

Manual posts show a gray **"Manual"** badge.

6.4 Visibility Toggle

Location: Admin → CMS → Settings → Features

The **"Show Instagram Feed on Storefront"** setting (`instagram_feed_visible`) controls whether the entire Instagram section appears on the storefront homepage. When set to `false`, the `<InstagramFeed />` component is not rendered.

7. Storefront Display

The `InstagramFeed` component (`apps/storefront/components/social/InstagramFeed.tsx`):

1. Fetches active posts from `GET /api/v1/cms/instagram-posts`
2. Displays them in a 6-column responsive grid

3. On hover, shows likes count and caption
4. Each post links to the actual Instagram post URL (or the main IG profile)
5. Includes a "Follow on Instagram" button linking to @narofashion2019

The homepage (`apps/storefront/app/page.tsx`) checks the `instagram_feed_visible` site setting before rendering the component.

8. Newsletter System

8.1 Subscriber Management

Subscription flow:

```
Storefront Homepage/Footer Form
|
▼
POST /api/v1/newsletter/subscribe
{ email: "user@example.com" }
|
▼
NewsletterSubscriber table
- Auto-generates unsubscribeToken
- If already exists & inactive: reactivates
- If already exists & active: returns "Already subscribed"
```

Unsubscribe flow:

```
Email Footer → "Unsubscribe" link
|
▼
GET /api/v1/newsletter/unsubscribe/{token}
|
▼
Sets isActive=false, unsubscribedAt=now()
|
▼
Storefront /unsubscribe page shows confirmation
```

8.2 Email Campaign Compose

Location: Admin → Newsletter → Compose

The compose page allows:

1. Selecting a template type (radio buttons)
2. Entering a subject line
3. Writing HTML body content with live preview
4. For NEW_ARRIVALS: auto-fetching products not yet sent
5. Saving as draft or sending immediately

8.3 Template Types

Type	Description	Auto-Population
NEW_ARRIVALS	Showcases new products	Yes — fetches products created since last NEW_ARRIVALS newsletter, excludes already-included products via NewsletterProduct join table

NEW_DEALS	Flash sales, promotions	No — admin writes content
TIPS	Fashion tips, style advice	No — admin writes content
CUSTOM	Free-form newsletter	No — admin writes content

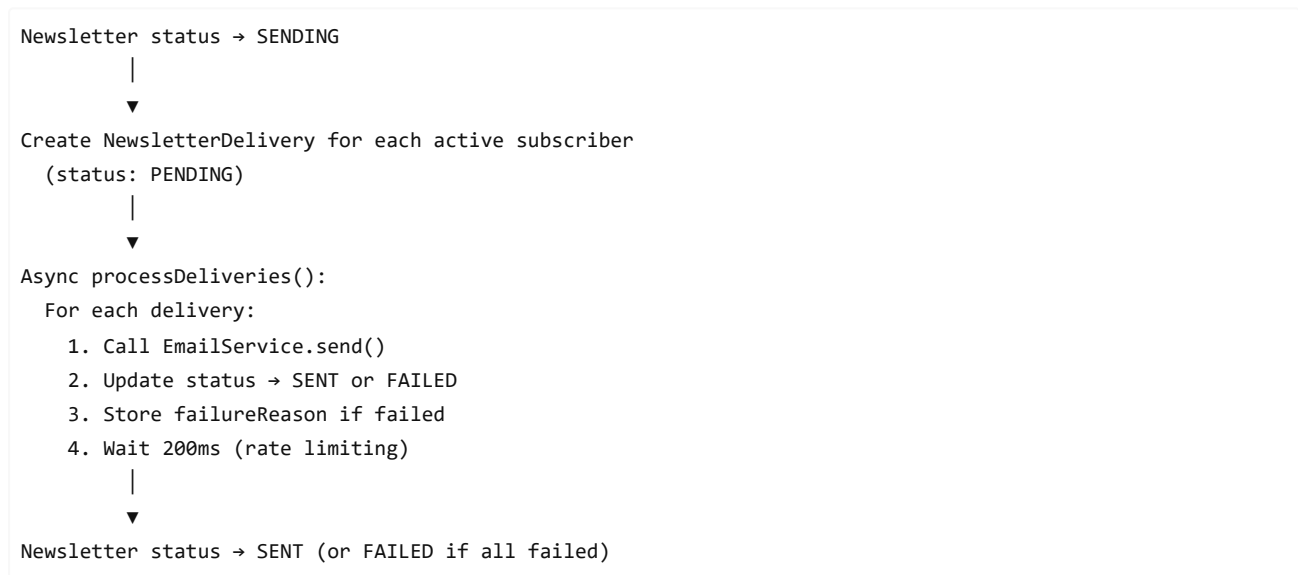
Email templates (Handlebars):

- `newsletter.hbs` — Generic: renders `{{bodyHtml}}` + unsubscribe link
- `newsletter-new-arrivals.hbs` — Product grid with images, prices, "Shop Now" buttons

All templates are wrapped in the master `layout.hbs` with Naro Fashion branding (black #1A1A1A + gold #D4AF37).

8.4 Delivery Tracking

When admin clicks "Send":



Admin can see:

- Total emails sent / failed / pending
- Delivery rate percentage
- Per-recipient failure reasons

8.5 Resending Failed Emails

Location: Admin → Newsletter → [newsletter detail] → "Resend Failed"

1. Resets all FAILED deliveries to PENDING
2. Sets newsletter status back to SENDING
3. Re-runs the async delivery process
4. Only retries previously failed recipients

9. Database Models

InstagramPost

id	String	@id @default(cuid())
caption	String?	


```

imageUrl      String
postUrl       String?
likes         Int      @default(0)
sortOrder     Int      @default(0)
isActive      Boolean  @default(true)
deletedAt     DateTime?
source        String   @default("MANUAL") // INSTAGRAM_API or MANUAL
isPinned      Boolean  @default(false)
instagramMediaId String? @unique           // Prevents duplicates
mediaType     String?  // IMAGE, VIDEO, CAROUSEL_ALBUM
postedAt      DateTime? // Original IG post date

```

NewsletterSubscriber

```

id           String   @id @default(cuid())
email        String   @unique
name         String?
isActive     Boolean  @default(true)
source       String   @default("STOREFRONT")
unsubscribeToken String @unique @default(cuid())
subscribedAt DateTime @default(now())
unsubscribedAt DateTime?

```

Newsletter

```

id           String   @id @default(cuid())
subject      String
bodyHtml     String
templateType String   @default("CUSTOM") // NEW_ARRIVALS, NEW_DEALS, TIPS, CUSTOM
status       String   @default("DRAFT")  // DRAFT, SENDING, SENT, FAILED
sentAt       DateTime?
createdById  String?

```

NewsletterDelivery

```

id           String   @id @default(cuid())
newsletterId String
subscriberId String
status       String   @default("PENDING") // PENDING, SENT, FAILED
sentAt       DateTime?
failureReason String?

```

NewsletterProduct

```

id           String   @id @default(cuid())
newsletterId String
productId    String
// Tracks which products were included to prevent duplicates

```

10. API Endpoints Reference

Instagram Posts (CMS)

Method	Endpoint	Auth	Description
GET	/cms/instagram-posts	Public	Active posts (ordered: API→Pinned→Manual)
GET	/cms/instagram-posts/admin	JWT	All posts for admin
POST	/cms/instagram-posts	JWT	Create manual post
PATCH	/cms/instagram-posts/:id	JWT	Update post
DELETE	/cms/instagram-posts/:id	JWT	Soft delete
PATCH	/cms/instagram-posts/:id/restore	JWT	Restore deleted
PATCH	/cms/instagram-posts/:id/pin	JWT	Toggle pin
POST	/cms/instagram-posts/sync	JWT	Trigger Instagram sync
GET	/cms/instagram-posts/deleted	JWT	Deleted posts

Newsletter

Method	Endpoint	Auth	Description
POST	/newsletter/subscribe	Public	Subscribe email
GET	/newsletter/unsubscribe/:token	Public	Unsubscribe by token
GET	/newsletter/subscribers	JWT	Paginated subscriber list
GET	/newsletter/subscribers/stats	JWT	Subscriber counts
GET	/newsletter/dashboard	JWT	Overall stats
POST	/newsletter	JWT	Create newsletter
GET	/newsletter	JWT	List newsletters
GET	/newsletter/:id	JWT	Newsletter detail
PATCH	/newsletter/:id	JWT	Update draft
DELETE	/newsletter/:id	JWT	Delete draft
POST	/newsletter/:id/send	JWT	Send newsletter
GET	/newsletter/:id/deliveries	JWT	Delivery stats
GET	/newsletter/:id/failed	JWT	Failed deliveries
POST	/newsletter/:id/resend-failed	JWT	Resend failed
GET	/newsletter/new-arrivals-preview	JWT	Products for NEW_ARRIVALS

11. Troubleshooting

Instagram Sync Fails

- **Check API logs** for error messages
- **Verify token:** Run in terminal:

```
curl "https://graph.facebook.com/v25.0/17841418108905851/media?fields=id&access_token=YOUR_TOKEN&limit=1"
```

- **Token expired?** Generate a new one via Graph API Explorer and exchange for long-lived
- **Wrong API URL?** Must use `graph.facebook.com` not `graph.instagram.com`

Token Expired (after 60 days)

1. Go to <https://developers.facebook.com/tools/explorer/>
2. Select Narofashion app
3. Generate new User Access Token with permissions
4. Exchange for long-lived token using the curl command in Section 4.5
5. Update `INSTAGRAM_ACCESS_TOKEN` in `.env`

Newsletter Emails Not Sending

- Check SMTP configuration in `.env` (`SMTP_HOST`, `SMTP_USER`, `SMTP_PASS`)
- If SMTP not configured, emails are logged only (check API console output)
- Check `NewsletterDelivery` records for `failureReason`

"Insufficient Developer Role" Error

- Ensure your Facebook account is an admin of the Meta Developer App
- The Instagram account must be a Professional/Business account
- The Facebook Page must be linked to the Instagram account

Appendix: Key File Locations

Purpose	File Path
Instagram sync service	<code>apps/api/src/cms/instagram.service.ts</code>
CMS service (ordering)	<code>apps/api/src/cms/cms.service.ts</code>
CMS controller (endpoints)	<code>apps/api/src/cms/cms.controller.ts</code>
Newsletter service	<code>apps/api/src/newsletter/newsletter.service.ts</code>
Newsletter controller	<code>apps/api/src/newsletter/newsletter.controller.ts</code>
Scheduler (cron jobs)	<code>apps/api/src/scheduler/scheduler.service.ts</code>
Email templates	<code>apps/api/src/notifications/templates/</code>
Admin IG posts page	<code>apps/admin/app/dashboard/cms/instagram-posts/page.tsx</code>
Admin newsletter pages	<code>apps/admin/app/dashboard/newsletter/</code>
Storefront IG feed	<code>apps/storefront/components/social/InstagramFeed.tsx</code>
Storefront subscribe forms	<code>apps/storefront/app/page.tsx</code> + <code>components/layout/Footer.tsx</code>
Unsubscribe page	<code>apps/storefront/app/unsubscribe/page.tsx</code>
Prisma schema	<code>packages/database/prisma/schema.prisma</code>
Environment config	<code>apps/api/.env</code>